IMPLEMENTATION PLAN

Wire the generator rules into the Model Selection Agent

Issue #33, as specified for this branch. Companion to Generator Selection Rules, Explained (15 October 2026) and to docs/architecture.md section 5.2.

Prepared 22 October 2026. No pipeline code is changed by this document.

This plan is the build sequence for three new pieces of behavior: a coded selection rule, an agent that turns that rule into a GeneratorOutput and logs every step, and tests that lock the result for Bank Marketing, Credit Card Default, Polish Bankruptcy, and IEEE-CIS. DCGAN is never a legal result for tabular data. WGAN-GP is the all-numeric result and is not implemented, so the result documents CTGAN as its fallback and does not train it.

1. Outcome

When the work is done, a caller can hand the agent a ModelSelectionInput and a run folder and receive a GeneratorOutput. The generator named in that output is the one a fixed procedure selected. The decision log contains one line per gate and per routing step, with the values that were used. The same data produces the same generator on every run.

- **src/agentic_gan/tools/selection.py** holds the rule. It is a pure function of a characteristics object. It does not read the disk, train a model, or import the run folder.
- **src/agentic_gan/agents/model_selection.py** builds those characteristics from the training split, calls the rule, writes each step with log_decision, and returns a GeneratorOutput.
- **tests/test_selection.py** drives the rule with characteristics for the four datasets, plus the gate failures. It does not fit a GAN.
- A small change in the orchestrator graph so a SKIPPED selection skips the quality check and the run continues. Existing stub tests keep the stub.

2. What is already in the repo

The branch starts from main at fb9049b. The pieces below stay as they are. The work sits on top of them.

Piece	Where	What it already does
Handoff types	schemas.py	ModelSelectionInput rejects a first attempt that carries a report, a retry that lacks one, and an attempt above max_attempts (default 3). GeneratorOutput on SUCCESS requires generator, rationale, checkpoint, loss history, and one synthetic set per fill level. SKIPPED needs only a reason. Optional fields may still be filled on a SKIPPED output.
Decision log	orchestrator/decisions.py	log_decision(agent, rule, inputs, outcome, rationale, run=...) appends one JSONL line. read_decisions loads it.
Quality loop	orchestrator/graph.py	After preprocessing, the graph calls model selection, then quality. A FAIL verdict calls model selection again with previous_report, up to three attempts, then continues training with gan_available false.
Stage contract	graph.execute	Any status other than SUCCESS is raised as a stage error, retried once, and then fails the run. SKIPPED is not honored.
Default agent	orchestrator/stubs.py	Model selection returns generator="stub" and does not look at the data. build_graph replaces only the quality stub with run_quality.
CTGAN tool	tools/ctgan.py	train_ctgan, sample_synthetic, save_model. The training table must have no numeric missing values and must not already contain is_synthetic. Sampling requests the positive class directly.

Piece	Where	What it already does
Run settings	RunConfig	min_rows defaults to 2,000. excluded_features, gan_bounds, and fill_levels exist. There is no min_steps field. gan_bounds defaults to an empty dict.
Datasets	utils/datasets.json	bank_marketing, default, and polish only. IEEE-CIS is not registered. The architecture note lists it as proposed and not in scope.
Profile	tools/preprocessing.py	profile_dataframe can count rows, duplicates, and missing values. The preprocessing script no longer writes profile.json. The preprocessing agent is still a stub that writes {}.

The quality agent is the pattern to copy. Its tools compute a result. The agent writes one log line per check. Selection should split the same way: the tool returns the steps, the agent logs them.

3. Decisions locked by this plan

These are the choices the implementation should follow. They reconcile the 46-rule explanation, the issue text, and the schema that is already merged.

3.1 The rule does not train, and it does not log

select_generator returns a SelectionResult. The result carries the generator, whether that generator can be trained today, the documented fallback, the comparator, a rationale, and an ordered tuple of DecisionStep records. Each step has a rule name, the inputs that were read, an outcome, and one sentence of rationale. The agent is the only caller of log_decision. Tests of the rule then need no run folder.

3.2 One procedure, applied to the table the generator would see

Gates run first. The temporal test runs next. Type routing runs only when the temporal test does not select TimeGAN. The first stopping gate ends the procedure. Later gates are still written into the step list as not reached, so the log shows the order rather than a single surviving rule.

3.3 Column names do not select TimeGAN

A column called Time, TransactionDT, day, or cohort is not a sequence. Temporal facts come from a structure table keyed by dataset name. The measured training frame can confirm those facts. It cannot invent them. An unknown dataset fails the temporal test and still goes through the type route.

3.4 What each gate is allowed to do

- **Size.** Training rows below config.min_rows, default 2,000, stop the procedure. Status is skipped. No generator.
- **Missing values.** Any unresolved NaN in a column the generator would see stops the procedure. Selection does not impute. Status is skipped, and the reason says repair has not finished.
- **Leakage.** If a column on that dataset's leakage list is still present, stop. Selection does not drop the column. Bank's list is duration. The other three lists are empty until a rule names a column.
- **Minority count.** Fewer than 500 minority rows does not stop the procedure. It forbids a generator trained on the minority class alone. A conditional generator may still be selected. The step records the count and the prohibition. This matches G-3, which blocks a separate minority generator.
- **Image and diffusion.** For every tabular dataset the step dcgan_excluded is written with outcome EXCLUDED before any positive choice. The returned generator is never dcgan, tablegan-as-image, or a diffusion model. If characteristics say the data is not tabular, the procedure stops as skipped and names no image generator.

3.5 The four temporal conditions

The explained rules say the temporal test has four conditions and that Polish fails condition d. They do not list the four names. This plan fixes the names so the code and the tests share one checklist. All four must pass, or TimeGAN is not selected.

Condition	Passes when	Fails when
entity_key	The structure table says each row is one entity, or a declared entity column is present.	No entity is declared. A row id such as TransactionID does not count.
time_index	A declared within-entity order exists. For Default, the six-month wide block is complete in the training columns.	The only time-like field is a single stamp on an independent row, or the monthly block is missing a column.
min_steps	Steps per entity are at least 5. Default has 6.	Fewer than 5 steps, or the step count cannot be checked.
repeated_observation	The axis is repeated measurement of that entity.	The axis is a forecast horizon (Polish year files), a campaign-date sort (Bank), or a transaction clock (ULB Time, IEEE-CIS TransactionDT).

Default can pass after ID has been dropped. Each source row is already one client, and the sequence lives in the wide monthly columns. The entity condition passes from the structure table, not from the ID column surviving in the cleaned file. If any of PAY_1 through PAY_6, BILL_AMT1 through BILL_AMT6, or PAY_AMT1 through PAY_AMT6 is absent, time_index fails and routing continues.

3.6 Type routing

- A non-target column that is categorical, or a binary missing-indicator, makes the table mixed. The primary generator is CTGAN. It is trainable.
- A table whose features are all numeric selects WGAN-GP. It is not trainable. The documented fallback is CTGAN. The comparator recorded on a CTGAN selection is WGAN-GP.
- TimeGAN's rationale states that it covers the monthly block only. Static columns are conditioning, not time steps. The documented fallback is CTGAN on the flattened wide table.
- The target column does not count as a categorical feature for this test. A binary target alone does not make a numeric table mixed.

3.7 Volume stays on the config grid

The explained rules describe augmentation as a target positive rate: 20, 35, and 50 percent, or 1, 5, and 10 percent for the ULB fraud set. The merged schema uses fill_levels, a share of the class gap, defaulting to 0.10, 0.25, 0.50, 0.75, and 1.00. This task uses fill_rows(level, baseline_train_positives, baseline_train_negatives) and the config grid. Changing the grid to the percentage ladder is a separate config decision and is not part of wiring the selector.

3.8 Training boundary

The agent trains only when the selected primary generator is CTGAN. It calls the existing CTGAN tool, writes the checkpoint and the loss history under the run's synthetic folder, and returns one SyntheticSet per fill level. Row counts come from fill_rows and the baseline class counts on PreprocessOutput, not from SplitOutput.

When the primary generator is TimeGAN or WGAN-GP, the agent does not train the fallback on that attempt. It returns SKIPPED. The generator field is the primary name. The reason and the rationale name the fallback and say why it was not trained. Training CTGAN in place of TimeGAN or WGAN-GP would hide the rule the log is supposed to defend.

3.9 Retries stay inside gan_bounds, and they are coded

G-1 allows an LLM to suggest hyperparameters inside config bounds on a retry. It does not allow an LLM to choose the generator. This task implements the retry with a deterministic adjuster, so a run does not need an API key and two runs with the same report take the same step. An LLM proposal can replace the adjuster later without changing the selector.

- Attempt 1 uses the CTGAN tool defaults: epochs 300, batch size 500, pac 10, discriminator steps 1. Each key present in gan_bounds is clamped into that interval. Batch size must stay divisible by pac. If the bounds make that impossible, the agent returns SKIPPED with that reason and does not train.
- Attempt 2 keeps the same generator. If a fidelity check failed, epochs move one step toward the upper bound when that bound allows it. A privacy or exact-duplicate failure does not increase epochs. If no bound allows a legal change, the same settings are repeated and the log says so.
- Attempt 3 is the named fallback. For a CTGAN primary the fallback is WGAN-GP, which is not trainable, so attempt 3 returns SKIPPED with the fallback documented. The graph then stops retrying.

4. How the four datasets resolve

Tests pass a DataCharacteristics object. They do not read the raw files. IEEE-CIS is not in the repository, and the fraud recommendation measured it on 16 October 2026. Those measured figures are the fixture. The ULB credit-card rules in the 46-rule document are a different dataset. They are not the IEEE-CIS expectation.

Dataset	Fixture the test passes	Result
Bank Marketing	Mixed types, about 27,000 training rows, 3,173 positives, no entity, duration absent.	CTGAN, trainable. TimeGAN excluded: no entity and no within-entity steps. DCGAN excluded.
Bank, leakage	The same table with duration still present.	Skipped at the leakage gate. No generator.
Bank, too small	The same mix with 1,500 rows.	Skipped at the size gate.
Credit Card Default	One row per client, the 18 monthly columns present, 6 steps, static columns separate, minority well above 500.	TimeGAN, not trainable. Scope is the monthly block. Documented fallback is CTGAN. DCGAN excluded.
Default, block broken	A monthly column removed. Sex, education, marriage, and repayment status remain categorical.	Temporal test fails. CTGAN by the mixed-type step.
Polish, pooled	More than one horizon in the table.	Skipped. Rule horizon_gate. PB-1: one horizon file, never pooled. No generator.
Polish, unrepaired	One horizon, unresolved missing values.	Skipped at the missing-value gate. No generator.
Polish, indicators kept	One horizon, no missing values, binary missing-indicator columns retained.	CTGAN, trainable. Comparator WGAN-GP, documented and not trained. TimeGAN excluded because the year is a horizon.
Polish, indicators dropped	One horizon, no missing values, 64 numeric ratios, no indicator columns.	WGAN-GP, not trainable. Documented fallback CTGAN. Not trained on this attempt.
IEEE-CIS, as measured	590,540 rows, 12,398 training fraud rows, mixed columns, unresolved missing values. TransactionDT is a clock. TransactionID is a row id.	Skipped at the missing-value gate. Temporal test would fail. DCGAN excluded.
IEEE-CIS, repaired	Same structure after missing values are resolved. Categorical columns remain.	CTGAN, trainable. Not WGAN-GP. TimeGAN excluded.
Minority gate	Mixed table, enough rows, 295 minority rows.	CTGAN still selected. The minority step forbids a class-only generator and does not stop the run.

The current Polish loader concatenates five horizon files and adds a cohort column, and the current Default loader drops ID. The selector does not change those loaders. On a real Polish run today, cohort with more than one value trips horizon_gate and the agent returns SKIPPED. A pooled table is refused until preprocessing pins one horizon file.

5. Selection module

src/agentic_gan/tools/selection.py exposes the types and one function. Suggested shape:

```
def select_generator(
    data: DataCharacteristics,
    *,
    min_rows: int = 2000,
    minority_min: int = 500,
    min_steps: int = 5,
) -> SelectionResult
```

DataCharacteristics fields:

- dataset, rows, target, minority_count, minority_rate, tabular.
- columns: name, kind in {numeric, categorical, binary}, missing count.
- entity_in_table, wide_steps, repeated_observation. These are filled from the structure table after the training columns have been checked. Tests may set them directly.
- leaky_columns: names that are both on the dataset leakage list and still present.
- horizons_pooled.

SelectionResult fields:

- status: selected or skipped.
- generator: ctgan, timegan, wgan_gp, or None.
- trainable: true only for ctgan.
- fallback: the documented next generator, or None when the procedure stopped at a gate.
- comparator: the other tabular generator, recorded and not trained.
- scope: a short restriction, used for TimeGAN's monthly block. Empty otherwise.
- rationale: the sentence that GeneratorOutput.rationale copies.
- steps: every gate and route, in order, including steps that were not reached.

Step rule names, in order:

dcgan_excluded, size_gate, horizon_gate, missing_gate, leakage_gate, minority_gate, temporal_test, type_routing, fallback.

temporal_test inputs include the four condition results, not only the final pass or fail. type_routing inputs include the column-kind counts and the generator those counts imply. fallback is written when a primary generator is not trainable, and again on attempt 3.

A structure table in the same module, DATASET_STRUCTURE, covers bank_marketing, default, polish, and ieee_cis. It records the leakage list, whether one row is one entity, the names of the wide temporal columns, the non-sequence time columns, and the fallback generator. describe_characteristics lives in the agent, not in this module, because it reads a frame and the run config.

6. Agent

src/agentic_gan/agents/model_selection.py exposes run_model_selection(request, run). That is the callable graph.execute already expects. The body is:

- Read only preprocess.baseline.train_path. Do not open the validation or test files.
- Drop source_row before any generator sees the frame. Leave the target in the frame. Confirm is_synthetic is absent.
- Count minority rows on the target. If that count disagrees with baseline_train_positives, raise. The schema then surfaces a stage error, which is a broken agent rather than a skipped selection.
- Resolve the positive label the way the quality agent already does: use config.positive_class when that value is in the column, and use 1 when the config says yes and the cleaned column is 0/1.
- Build DataCharacteristics. Measured facts from the frame are rows, missing counts, and column kinds. Kinds come from utils/datasets.json discrete columns when the dataset is registered, and from dtype otherwise. Binary indicator columns are kind binary. Temporal facts come from DATASET_STRUCTURE, confirmed against the columns actually present.
- Call select_generator with config.min_rows. Log every step with agent model_selection.
- If the result is skipped, or the generator is not trainable, return GeneratorOutput(status=SKIPPED, reason=..., generator=..., rationale=..., warnings=...). Do not invent a checkpoint path.
- If the generator is CTGAN, clamp hyperparameters, train, sample once per fill level, and return SUCCESS.

6.1 Files a successful CTGAN attempt writes

- synthetic/ctgan_attempt_{n}.pkl via save_model. This is checkpoint_path.
- synthetic/ctgan_loss_attempt_{n}.csv via the tool's log_path. This is loss_history_path.
- synthetic/fill_{level}_attempt_{n}.csv for each config fill level. rows equals fill_rows for that level. The file includes is_synthetic = 1.
- sample_synthetic is asked for the positive class only, with drop_duplicates_of set to the training columns the model saw.

Tests of the agent monkeypatch train_ctgan, sample_synthetic, and save_model. The default test run must not fit a neural net. A separate marked test may call the real tool later. It is not required to close this issue.

6.2 Attempt 2 adjustment

The agent reads previous_report.levels and collects the hard checks that failed. The mapping is fixed:

Failed check name contains	Adjustment	When there is no room
column_distance, correlation, ks, js, fidelity	Increase epochs toward the upper gan_bounds entry, without passing it.	Log retry_unchanged and keep the previous epochs.
dcr, duplicate, privacy	Do not increase epochs.	Log no privacy-safe adjustment.
Any other hard failure	No change.	Log retry_unchanged.

The proposal is its own decision-log line, rule hyperparameter_retry, with the old value, the new value, and the bound. Values outside gan_bounds are not emitted. An empty gan_bounds means the defaults are already the only legal settings, so attempt 2 repeats them.

7. Orchestrator changes required to use the output

The issue names three files. The current graph cannot consume a SKIPPED GeneratorOutput, so one companion change belongs in this task. Without it, Default, unrepaired Polish, unrepaired IEEE-CIS, a too-small table, and a WGAN-GP selection all crash the run.

7.1 Keep the stub unless it is still the stub

Inside build_graph, after the existing quality replacement, install run_model_selection only when the model-selection entry is a StubAgent. test_quality_agent_in_graph monkeypatches default_agents and supplies its own generator fixture. An unconditional overwrite would destroy that fixture. Tests that pass agents=default_agents() explicitly keep every stub, which is what test_orchestrator.py relies on.

7.2 Honor SKIPPED from model selection only

execute stays strict for every other stage. The model-selection node, when the returned status is SKIPPED:

- Finishes the stage with finish(SKIPPED, reason). The stage has already been started, so RunState.skip is the wrong call. skip is for a stage that never started.
- Does not raise, and does not use the error retry.
- Registers the output. A SKIPPED output with no paths adds nothing to the manifest, which register already allows.
- Sets gan_available false with the selection reason.
- Marks quality_check with skip, because that stage never started. is_done treats SKIPPED as done, so the run can still complete.
- Routes to training with no accepted synthetic sets.

training currently reads state["quality_check"] unconditionally, and evaluation passes that report through. Both need to tolerate a missing report: accepted synthetic sets are empty, and EvaluationInput.quality is already optional. QualityInput requires a SUCCESS generator, so the quality node must not be called on a SKIPPED selection.

Add one graph test. A model-selection agent that returns SKIPPED produces a complete run, gan_available false, quality_check skipped with zero attempts, training succeeded, and a decision-log line for the selection. The existing pass, retry, and exhaustion tests stay on the stub generator and must still pass.

8. Tests in tests/test_selection.py

Two layers live in that file. The first calls select_generator and never builds a run. The second calls run_model_selection with a temporary run directory and a monkeypatched CTGAN tool.

8.1 Rule tests

- One test per row of the dataset table in section 4.
- Each test asserts generator, status, trainable, fallback, and the outcome of the step that decided it.
- Each test asserts that no step has generator dcgan and that dcgan_excluded is present with outcome EXCLUDED for every tabular fixture.
- The Default pass asserts all four temporal conditions passed, and that scope mentions the monthly block.

- The IEEE-CIS repaired fixture asserts the temporal conditions failed on time_index and repeated_observation even though a column named TransactionDT is in the inputs.
- The minority fixture asserts minority_gate outcome FORBIDDEN_CLASS_ONLY and a later type_routing outcome of ctgan.
- A not-reached step is present after a stopping gate, with outcome NOT_REACHED, so the order is visible.
- Calling the function twice on the same object returns equal results.

8.2 Agent tests

- Bank-shaped frame, duration absent, CTGAN monkeypatched to write a fake checkpoint, a one-row loss file, and a frame of the requested length. Expect SUCCESS, generator ctgan, one synthetic set per fill level, and rows equal to fill_rows.
- The decision log contains dcgan_excluded, the four gates, temporal_test, and type_routing.
- The agent was not given the validation path's contents. The test can pass a missing validation path only if the schema's PreprocessOutput requires the path to exist as a string. Point val and test at an empty unread file and assert the monkeypatched trainer received only the training columns.
- Duration present: SKIPPED, leakage_gate FAIL, train_ctgan not called.
- All-numeric fixture: SKIPPED, generator wgan_gp, rationale contains the CTGAN fallback, train_ctgan not called.
- Attempt 2 with a failed fidelity check and gan_bounds epochs (100, 600) calls the trainer with a larger epoch count than attempt 1, still inside the bound.
- Attempt 2 with empty gan_bounds calls the trainer with the same defaults and logs retry_unchanged.
- Attempt 3 after a CTGAN primary returns SKIPPED, names wgan_gp as the fallback, and does not call train_ctgan.
- A first attempt that includes previous_report is not constructed: ModelSelectionInput already rejects it. The test only needs to show the agent reads a legal retry.

9. Build order

Each step is reviewable on its own. Tests for a step land in the same change as the code.

Step	Add	Check before moving on
1	Data classes, DATASET_STRUCTURE, select_generator.	Section 8.1 tests pass. This module does not import torch, log_decision, or the run folder.
2	describe_characteristics and the SKIPPED return path in the agent, including decision logging.	Leakage, size, missing, TimeGAN, and WGAN-GP agent tests pass. Log lines match the steps.
3	CTGAN success path: clamp bounds, train, sample, fill_rows.	Bank SUCCESS test passes with the tool monkeypatched. Synthetic row counts match fill_rows. source_row never enters the trainer.
4	Attempt 2 adjuster and attempt 3 fallback.	The two retry tests and the attempt-3 test pass. No value leaves gan_bounds.
5	Graph: install the real agent only over a stub, honor SKIPPED, let training and evaluation continue.	New graph test passes. test_orchestrator.py and test_quality_agent.py still pass.

10. Done when

- select_generator is deterministic and contains the only copy of the gate order.
- Bank with a legal table selects CTGAN. Default with a complete monthly block selects TimeGAN and does not train. Polish all-numeric selects WGAN-GP, documents CTGAN, and does not train. Repaired mixed IEEE-CIS selects CTGAN. Unrepaired IEEE-CIS stops at missing values.
- DCGAN is absent from every returned generator and present in every tabular log as excluded.
- A SKIPPED selection completes the rest of the pipeline with gan_available false and no quality-check attempt.
- A CTGAN selection can return SUCCESS with artifacts the quality agent is able to read, using the existing tool.
- A quality FAIL on that output causes the graph to call the agent again. Attempt 2 changes only settings inside gan_bounds. Attempt 3 documents the unbuilt fallback and returns SKIPPED.

- The decision log is sufficient to reconstruct the choice: dataset, row count, minority count, each gate, each temporal condition, the rule that fired, and the exclusions.

11. Left for later

These are real parts of the 46 rules and of architecture section 5.2. They are not required to close this issue, and this change should not half-implement them.

- A TimeGAN tool, the wide-to-long reshape, and the static-column join. Until those exist, Default remains a documented TimeGAN selection.
- A WGAN-GP or TableGAN tool. Until one exists, all-numeric data remains a documented WGAN-GP selection with CTGAN named as the fallback.
- Training the fallback on attempt 1 so an all-numeric dataset still produces synthetic rows. That would answer a different question from the one the issue asks.
- The percentage augmentation ladders, dataset sanity lists, and the extra temporal and privacy checks. Those belong to the quality gate and the run config.
- Registering IEEE-CIS in datasets.json, subsampling it, and accepting its licence constraints.
- Changing the Polish loader to pin one horizon file, and changing the Default loader. The selector reacts to the table it is given.
- An LLM hyperparameter proposal. The coded adjuster is the stand-in that keeps G-1.
- SMOTE as a comparator. The training stage already has that arm.

12. Risks to watch while building

- SUCCESS without files will fail schema validation. Any trainable path must write the checkpoint, the loss file, and every synthetic set before returning.
- Importing tools/ctgan.py loads torch. The selection module and the rule tests must not import it. The agent may import it inside the CTGAN branch so a SKIPPED path stays light.
- Bank preprocessing maps the label yes to 1. Sampling with positive class yes against a 0/1 column yields an empty file. Resolve the label from the frame.
- fill_rows of a balanced fixture is 0, and SyntheticSet.rows must be at least 1. Agent tests need a real class gap. A zero gap should return SKIPPED with an explicit reason rather than an invalid SUCCESS.
- execute retries a raised error once. A bug in characteristic building will run the agent twice and can look like a quality retry. Keep the selection exceptions specific.
- Installing the real agent over every default_agents() result will break the quality graph test. The StubAgent check in section 7.1 is there for that reason.

Sources: Generator Selection Rules, Explained (46 rules, 15 October 2026); the issue statement for wiring those rules into the agent; docs/architecture.md sections 5.2, 7, and 8; docs/fraud_dataset_recommendation.md for the IEEE-CIS measurements; the code on this branch at the time of writing, including the empty model_selection.py, the CTGAN tool, and the quality-gate loop in orchestrator/graph.py.